Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЕЕ ЗАДАНИЕ №2

«Разработка консольного приложения с применением ООП»

ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки информации и управления»

Выполнил: студент гр. ИУ5-22Б _____ (Маслов А.О.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Москва, 2026 г

Целью данной работы является разработка консольного приложения на языке программирования C# с использованием объектно-ориентированного подхода и базы данных SQLite для хранения, обработки и анализа данных. В процессе выполнения задания необходимо было реализовать полноценную архитектуру приложения, включающую классы-модели, работу с базой данных, пользовательский интерфейс в виде консольного меню и механизм формирования отчётов.

В качестве предметной области был выбран вариант 19 - «Магазины и заказы». Данная предметная область описывает взаимодействие между магазинами и заказами, оформляемыми в них. Основной особенностью является связь типа «один ко многим», при которой один магазин может иметь несколько заказов, а каждый заказ относится только к одному магазину.

Для реализации данной модели была разработана структура базы данных, включающая две таблицы: Shop и Order. Таблица Shop является справочной и содержит информацию о магазинах, включая уникальный идентификатор и название. Таблица Order является основной и хранит данные о заказах: идентификатор заказа, внешний ключ, связывающий заказ с магазином, название заказа и сумму. Связь между таблицами реализована через внешний ключ ShopId, который ссылается на поле Id таблицы Shop. Такая структура обеспечивает целостность данных и позволяет выполнять объединённые запросы.

Особое внимание было уделено ограничениям данных. Для числового поля Amount, представляющего сумму заказа, было введено ограничение: значение не может быть отрицательным. Данная проверка реализована на уровне модели данных с использованием свойства с set-аксессором, генерирующим исключение при попытке присвоения некорректного значения.

Для представления сущностей предметной области были реализованы классы Shop и Order. Каждый класс содержит свойства, соответствующие полям таблицы, конструкторы (с параметрами и по умолчанию) и переопределённый метод ToString() для удобного вывода информации. Использование свойств вместо обычных полей позволяет инкапсулировать данные и реализовать дополнительную логику, например проверку корректности значений.

Работа с базой данных реализована в отдельном классе DatabaseManager, который инкапсулирует все операции взаимодействия с SQLite. В данном классе реализованы методы для создания таблиц, импорта данных из CSV-файлов, получения списка записей, добавления новых записей, удаления и поиска по идентификатору. Такой подход позволяет отделить логику работы с базой данных от остальной части приложения и делает код более структурированным и удобным для сопровождения.

Пользовательское взаимодействие с программой реализовано через консольное меню, работающее в цикле. Меню предоставляет пользователю возможность просматривать список магазинов и заказов, добавлять новые записи, удалять существующие и вызывать отчёты. Ввод данных

сопровождается проверкой корректности, что предотвращает ошибки выполнения программы и повышает её устойчивость.

Для формирования отчётов был реализован класс ReportBuilder, использующий паттерн Fluent Interface. Данный паттерн позволяет настраивать параметры отчёта в виде цепочки вызовов методов, где каждый метод возвращает текущий объект. В результате формируется гибкий и удобный механизм построения отчётов. Внутри класса используется выполнение SQL-запроса через DatabaseManager, после чего результат форматируется в строку с помощью StringBuilder.

В рамках дополнительного задания был реализован терминальный метод SaveToFile, позволяющий сохранять сформированный отчёт в текстовый файл. Этот метод использует результат работы метода построения отчёта и записывает его в файл по указанному пути. Данная функциональность расширяет возможности программы и позволяет сохранять результаты работы для дальнейшего использования.

Также в программе реализованы SQL-запросы для формирования отчётов, включая выборку данных с использованием оператора JOIN, подсчёт количества записей с помощью COUNT, а также вычисление среднего значения с использованием функции AVG. Это позволяет получать сводную информацию по данным и анализировать их.

Таким образом, в ходе выполнения работы было разработано полноценное консольное приложение, реализующее основные принципы объектно-ориентированного программирования, работу с реляционной базой данных и формирование отчётов. Полученные навыки могут быть использованы при разработке более сложных информационных систем и приложений, работающих с данными.

```
+----------------------+
|        Shop          |
+----------------------+
| Id : int [PK]        |
| Name : string        |
+----------------------+

          1
          |
          |
          v
          M

+------------------------------+
|           Order              |
+------------------------------+
| Id : int [PK]                |
| ShopId : int [FK -> Shop.Id] |
| Name : string                |
| Amount : int [>= 0]          |
+------------------------------+
```

Рисунок 1 – Схема модели данных

Работоспособность программы:

```
1 – Показать магазины
2 – Показать заказы
3 – Добавить заказ
4 – Удалить заказ
5 – Отчёт
0 – Выход
Выбор: 1
[1] Пятёрочка
[2] Дикси
[3] Магнит
[4] Перекрёсток
```

```
1 – Показать магазины
2 – Показать заказы
3 – Добавить заказ
4 – Удалить заказ
5 – Отчёт
0 – Выход
Выбор: 2
[1] Заказ продуктов, магазин #1, сумма: 1250 руб.
[2] Заказ бытовой химии, магазин #1, сумма: 890 руб.
[3] Заказ напитков, магазин #1, сумма: 540 руб.
[4] Заказ овощей, магазин #2, сумма: 760 руб.
[5] Заказ молочных продуктов, магазин #2, сумма: 430 руб.
[6] Заказ сладостей, магазин #2, сумма: 680 руб.
[7] Большой заказ продуктов, магазин #3, сумма: 3450 руб.
[8] Заказ для офиса, магазин #3, сумма: 5200 руб.
[9] Заказ посуды, магазин #3, сумма: 2100 руб.
[10] Заказ фруктов, магазин #4, сумма: 980 руб.
[11] Заказ выпечки, магазин #4, сумма: 350 руб.
[12] Заказ готовой еды, магазин #4, сумма: 1450 руб.
[15] Пирожки, магазин #2, сумма: 250 руб.
[16] Посуда, магазин #1, сумма: 3990 руб.
[17] Тарелка, магазин #1, сумма: 990 руб.
```

```
1 – Показать магазины
2 – Показать заказы
3 – Добавить заказ
4 – Удалить заказ
5 – Отчёт
0 – Выход
Выбор: 3
ID магазина: 1
Название: Пельмени
Сумма: 329
Добавлено!
```

```
1 – Показать магазины
2 – Показать заказы
3 – Добавить заказ
4 – Удалить заказ
5 – Отчёт
0 – Выход
Выбор: 2
[1] Заказ продуктов, магазин #1, сумма: 1250 руб.
[2] Заказ бытовой химии, магазин #1, сумма: 890 руб.
[3] Заказ напитков, магазин #1, сумма: 540 руб.
[4] Заказ овощей, магазин #2, сумма: 760 руб.
[5] Заказ молочных продуктов, магазин #2, сумма: 430 руб.
[6] Заказ сладостей, магазин #2, сумма: 680 руб.
[7] Большой заказ продуктов, магазин #3, сумма: 3450 руб.
[8] Заказ для офиса, магазин #3, сумма: 5200 руб.
[9] Заказ посуды, магазин #3, сумма: 2100 руб.
[10] Заказ фруктов, магазин #4, сумма: 980 руб.
[11] Заказ выпечки, магазин #4, сумма: 350 руб.
[12] Заказ готовой еды, магазин #4, сумма: 1450 руб.
[15] Пирожки, магазин #2, сумма: 250 руб.
[16] Посуда, магазин #1, сумма: 3990 руб.
[17] Тарелка, магазин #1, сумма: 990 руб.
[19] Пельмени, магазин #1, сумма: 329 руб.
```

```
1 – Показать магазины
2 – Показать заказы
3 – Добавить заказ
4 – Удалить заказ
5 – Отчёт
0 – Выход
Выбор: 4
ID заказа: 19
Удалено!

1 – Показать магазины
2 – Показать заказы
3 – Добавить заказ
4 – Удалить заказ
5 – Отчёт
0 – Выход
Выбор: 2
[1] Заказ продуктов, магазин #1, сумма: 1250 руб.
[2] Заказ бытовой химии, магазин #1, сумма: 890 руб.
[3] Заказ напитков, магазин #1, сумма: 540 руб.
[4] Заказ овощей, магазин #2, сумма: 760 руб.
[5] Заказ молочных продуктов, магазин #2, сумма: 430 руб.
[6] Заказ сладостей, магазин #2, сумма: 680 руб.
[7] Большой заказ продуктов, магазин #3, сумма: 3450 руб.
[8] Заказ для офиса, магазин #3, сумма: 5200 руб.
[9] Заказ посуды, магазин #3, сумма: 2100 руб.
[10] Заказ фруктов, магазин #4, сумма: 980 руб.
[11] Заказ выпечки, магазин #4, сумма: 350 руб.
[12] Заказ готовой еды, магазин #4, сумма: 1450 руб.
[15] Пирожки, магазин #2, сумма: 250 руб.
[16] Посуда, магазин #1, сумма: 3990 руб.
[17] Тарелка, магазин #1, сумма: 990 руб.
```

Дополнительное задание для 2 группы:

```
1 - Показать магазины
2 - Показать заказы
3 - Добавить заказ
4 - Удалить заказ
5 - Отчёт
0 - Выход
Выбор: 5
Отчет сохранен в файл C:\Users\Артём\source\repos\homework_2\homework_2\отчёт.txt
```

отчёт.txt | program.cs | homework_2.csproj | DatabaseManager.cs | shops.cs

```
=== Все заказы ===
Название                                Сумма
----------------------------------------------
Заказ продуктов                         1250
Заказ бытовой химии                     890
Заказ напитков                          540
Заказ овощей                            760
Заказ молочных продуктов                430
Заказ сладостей                         680
Большой заказ продуктов                 3450
Заказ для офиса                         5200
Заказ посуды                            2100
Заказ фруктов                           980
Заказ выпечки                           350
Заказ готовой еды                       1450
Пирожки                                 250
Посуда                                  3990
Тарелка                                 990
```